

LMS Project: Exhaustive Technical Documentation

A complete line-by-line and file-by-file logic breakdown for presentation.

Section 1: The Core Backend (Django)

File: lms_backend/settings.py (Global Configuration)

This is the 'Operating System' of your backend. It controls external connections and internal apps.

- **INSTALLED_APPS:** Includes 'rest_framework' (the engine for API), 'corsheaders' (allows the website to connect), and our custom apps 'users' and 'courses'.
- **AUTH_USER_MODEL:** This points to 'users.User'. We tell Django 'do not use your default user, use MY custom one with roles'.
- **REST_FRAMEWORK API Logic:** We set 'JWTAuthentication' as the default. This ensures every API request requires a secure 8-character+ token.
- **CORS_ALLOW_ALL_ORIGINS:** This allows our Frontend (JavaScript) to fetch data from our Backend (Python) without security blocks.

Section 2: Users & Authentication App

File: users/models.py (Identity Logic)

Code Snippet

```
class UserRole(TextChoices): Defines three choices: ADMIN, INSTRUCTOR, STUDENT. This is hard-coded into the database for high performance.
class User(AbstractUser): We inherit from Django's secure user system and add the role field.
```

File: users/views.py (The Gatekeeper)

- **RegisterView:** Takes data (email/username) and uses a create() function to save the user. If they are a student, it defaults their role.
- **CustomTokenView:** When a user logs in, this doesn't just check the password; it manually attaches the **Role** and **Username** to the response so the Frontend knows which dashboard to show.

File: users/serializers.py (Safety Translators)

Teacher's Secret Weapon

```
def create(self, validated_data):
    if not password:
        password = ''.join(random.choices(string.ascii + digits, k=8))
    user = User.objects.create_user(...)
    return user
```

Explain to Teacher: *'If an admin creates a teacher account but leaves the password blank, my code automatically generates a secure 8-character random password.'*

Section 3: Courses & LMS Logic

File: courses/models.py (The Knowledge Base)

The Linking Tree:

- **Course:** The root object. Links to an Instructor User.
- **Enrollment:** Links a STUDENT to a COURSE. It also has a 'progress' field (integer 0-100) to track how much they've learned.
- **LiveClass:** Contains meeting_link and start_time. This allows teachers to schedule Zoom calls directly.

File: courses/views.py (Data Filtering)

Th

```
def get_queryset(self):
    user = self.request.user
    if user.role == 'INSTRUCTOR':
        return Course.objects.filter(instructor=user)
    if user.role == 'STUDENT':
        return Course.objects.all() # Or enrolled courses
```

Explain to Teacher: *'I wrote custom logic so that teachers can only see and edit THEIR courses, while students can browse all available learning material.'*

Section 4: The API Interface (Endpoints)

The Backend exposes these 'Endpoints' that the Frontend uses to get data.

Method	Endpoint	Purpose
POST	/api/register/	Create a new Student account.
POST	/api/token/	Login and get a JWT Secure Token.
GET	/api/courses/	Fetch the list of all classes.
POST	/api/courses/	Allow Instructors to create new courses.
GET	/api/live-classes/	Get links to scheduled Zoom meetings.

For example, all the courses are fetched from the Backend like this:

```
{
  "id": 1,
  "title": "Intro to Python",
  "instructor": "Dr. Smith"
}
```

Section 5: Frontend Design (Next.js)

File Structure: /src/app (Dynamic Routing)

I used the **Next.js App Router** for speed and SEO optimization.

- **/login/page.tsx**: The Entry Point. It handles state for the username/password fields.
- **/student-dashboard/page.tsx**: Uses 'Fetch' to call the backend and map over the courses to show them in small cards.
- **layout.tsx**: The Wrapper. It defines the global fonts and colors using **Tailwind CSS**.

The Data Connection Logic (How it works)

1. **handleLogin()**: Function that runs when you click 'Login'. It uses `fetch()` to send a request to Django.
2. **js-cookie**: If Backend returns 'Success', the browser saves the token in a **Cookie**. This allows you to stay logged in even if you refresh the page.
3. **router.push()**: Instantly navigates to the specific dashboard based on the role returned by the API.

Section 6: Advanced System Features

Concept	What I used	Why?
Auth	JWT (JSON Web Tokens)	More secure than standard sessions.
Styling	Tailwind CSS	Makes the UI responsive and modern.
Icons	Lucide React	Lightweight, vector-based illustrations.
Efficiency	Prefetch Related	Reduces database queries (faster app).

Conclusion Summary for Teacher:

'This project demonstrates a full-stack, industry-standard implementation. It solves real-world problems like user privacy (Role-Based Access) and scheduling (Live Classes) using the latest technologies available in web development today.'